计 算 机 体 系 结 构

第四讲

计算机科学与技术学院
舒燕君

Recap

- 计算机体系结构的发展
  - ✓并行体系结构的发展
  - ✓处理器技术的发展
- 成本与价格：学习曲线
- 基准测试程序
- 量化分析的基本原则
  - ✓大概率优先
  - ✓**Amdal**定律

1.2.6 并行处理技术的发展

- 计算机系统结构的Flynn分类法
  - Flynn分类法 按照指令和数据的关系，把计算机系统的结构分为4类
    - 单指令流单数据流SISD
      (Single Instruction stream Single Data stream)
    - 单指令流多数据流SIMD
      (Single Instruction stream Multiple Data stream)
    - 多指令流单数据流MISD
      (Multiple Instruction stream Single Data stream)
    - 多指令流多数据流MIMD
      (Multiple Instruction stream Multiple Data stream)

1.2.6 并行处理技术的发展

2 提高并行性的技术途径

三种途径：

（1）时间重叠

引入时间因素，让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部分，以加快硬件周转而赢得速度。

（2）资源重复

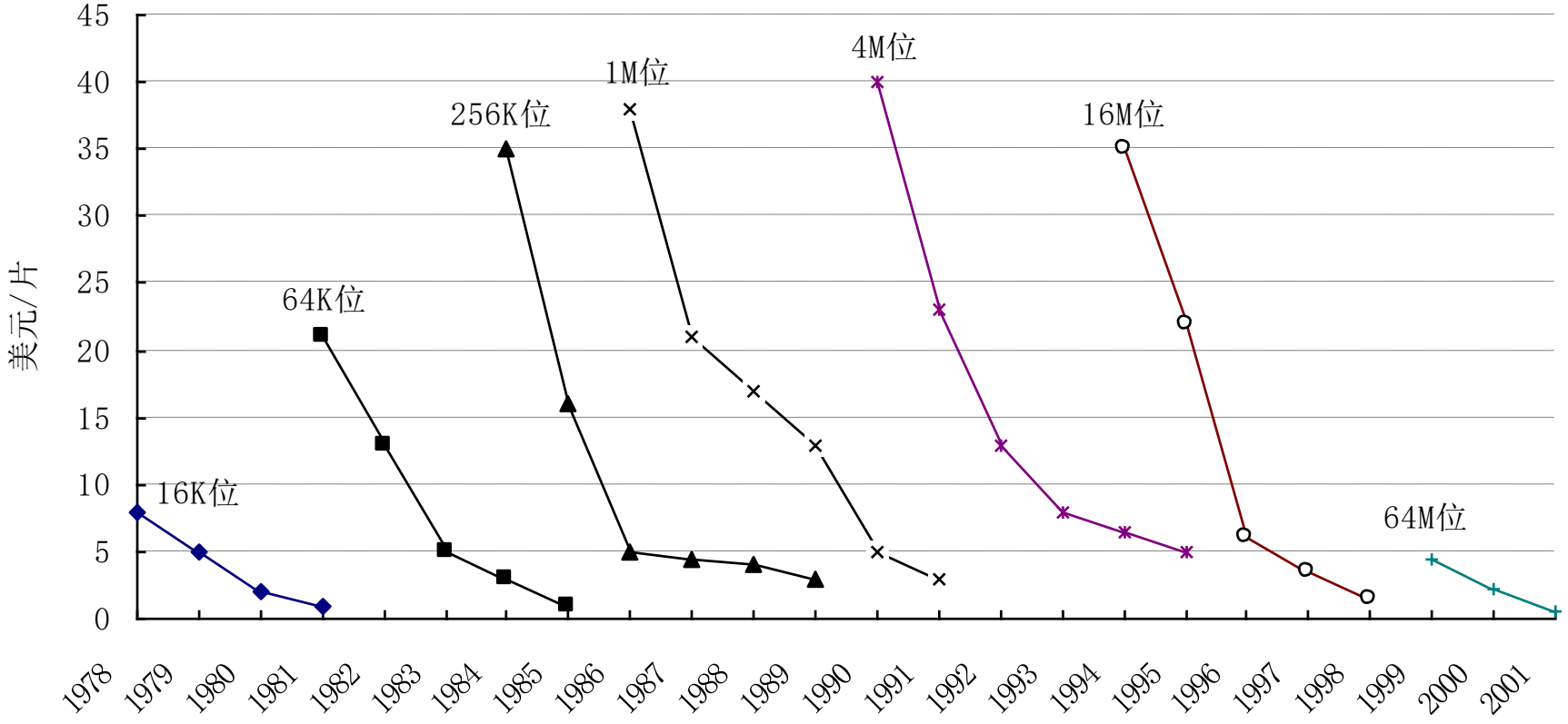
引入空间因素，以数量取胜。通过重复设置硬件资源，大幅度地提高计算机系统的性能。

（3）资源共享

这是一种软件方法，它使多个任务按一定时间顺序轮流使用 同一套硬件设备。

成本-时间因素：学习曲线

- 产品价格随时间变化的特性，就是价格随时间下降的趋势



1.3.2 基准测试程序

- 性能与测试程序的执行时间相关，那么用什么做测试程序呢?
- 五类测试程序
  - 真实程序
  - 修正的（或者脚本化）应用程序
  - 核心程序
  - 小测试程序
  - 合成测试程序
- 测试程序包(组件, benchmark suites)
  - 选择一组各个方面有代表性的测试程序组成
  - 尽可能全面地测试了一个计算机系统的性能

1.3.3 量化设计的基本原则

1．大概率事件优先原则
   – 追求全局的最优结果

2．Amdahl定律
   – 系统性能加速比，受限于该部件在系统中所占的重要性
   – 可以定量计算

3．程序的局部性原理
   – 程序执行时所访问存储器在时-空上是相对地簇聚
   – 这种簇聚包括指令和数据两部分

第 1 章 计算机系统量化分析基础

1.1 计算机体系结构的概念

1.2 计算机体系结构的发展

1.3 计算机系统设计和分析

1.3 计算机系统设计和分析

- 1.3.1 成本与价格
- 1.3.2 基准测试程序
- 1.3.3 量化设计的基本原则

程序局部性

- 程序访问地址的分布不是随机的，而是相对地簇聚
  - 包括程序的时间局部性和程序的空间局部性
- 程序的时间局部性
  - 程序即将用到的信息很可能就是目前正在使用的信息
- 程序的空间局部性
  - 程序即将用到的信息很可能与目前正在使用的信息在空间上相邻或者临近
- 其他局部性
  - 生产-消费局部性

局部性举例

```
sum = 0;
for (i = 0; i < n; i++)
    sum += a[i];
return sum;
```

- 对数据的引用
  - 顺序访问数组元素 (步长为1的引用模式) — 空间局部性
  - 变量sum在每次循环迭代中被引用一次 — 时间局部性
- 对指令的引用
  - 顺序读取指令 — 空间局部性
  - 重复循环执行for循环体 — 时间局部性

1.3.3 量化设计的基本原则

4. CPU性能公式

- 执行一个程序所需的CPU时间

  CPU时间 = 执行程序所需的时钟周期数×时钟周期时间

  其中：时钟周期时间是系统时钟频率的倒数。

- 每条指令执行的平均时钟周期数CPI

  CPI = 执行程序所需的时钟周期数 / IC

  IC：所执行的指令条数

- 程序执行的CPU时间可以写成

  CPU时间 = IC ×CPI ×时钟周期时间

  时钟周期时间：取决于硬件实现技术和计算机组成

  CPI：取决于计算机组成和指令系统的结构；

  IC：取决于指令系统的结构和编译技术

CPU性能公式

- 假设计算机系统有n 种指令，其中第i 种指令的处理时间为CPI_i，在程序中第i 种指令出现的次数为IC_i。

$$
\begin{aligned}
T_{CPU} &= \sum (IC_i \times CPI_i) / f \\
&= \sum (IC_i \times CPI_i) \times T_{CLK}
\end{aligned}
$$

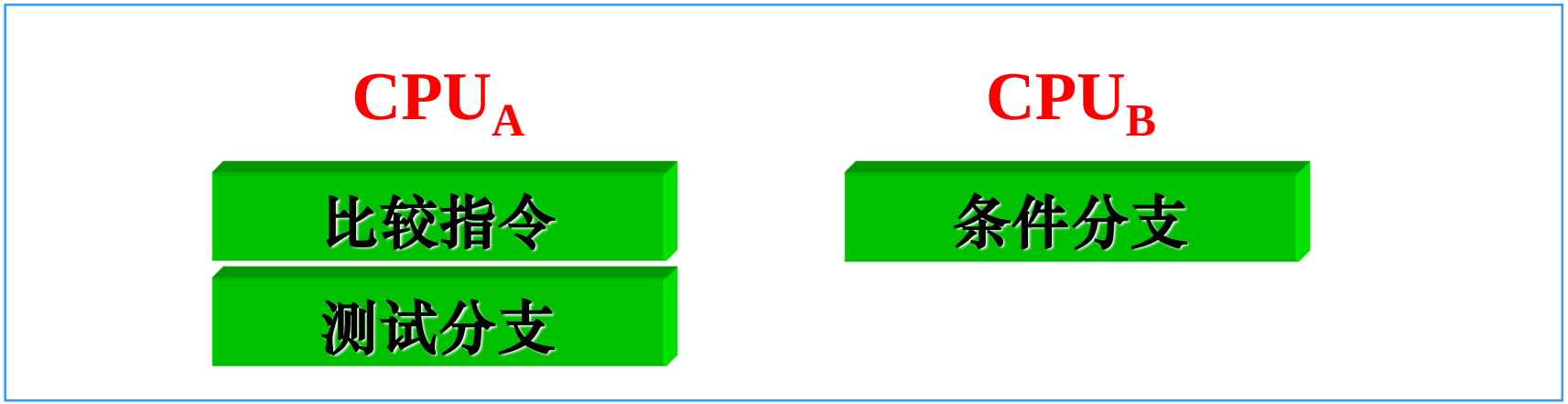
$$
\begin{aligned}
CPI &= \sum (IC_i \times CPI_i) / IC \\
&= \sum [(IC_i / IC) \times CPI_i]
\end{aligned}
$$

其中： IC_i / IC **反映了第i种指令在程序中所占的比例。**

CPU性能公式

例：假设我们考虑分支指令的两种不同设计方法如下：

- CPU_A：通过比较指令设置条件码，然后测试条件码进行分支。
- CPU_B：在分支指令中包括条件比较过程。



CPU性能公式

- 哪一个CPU更快?

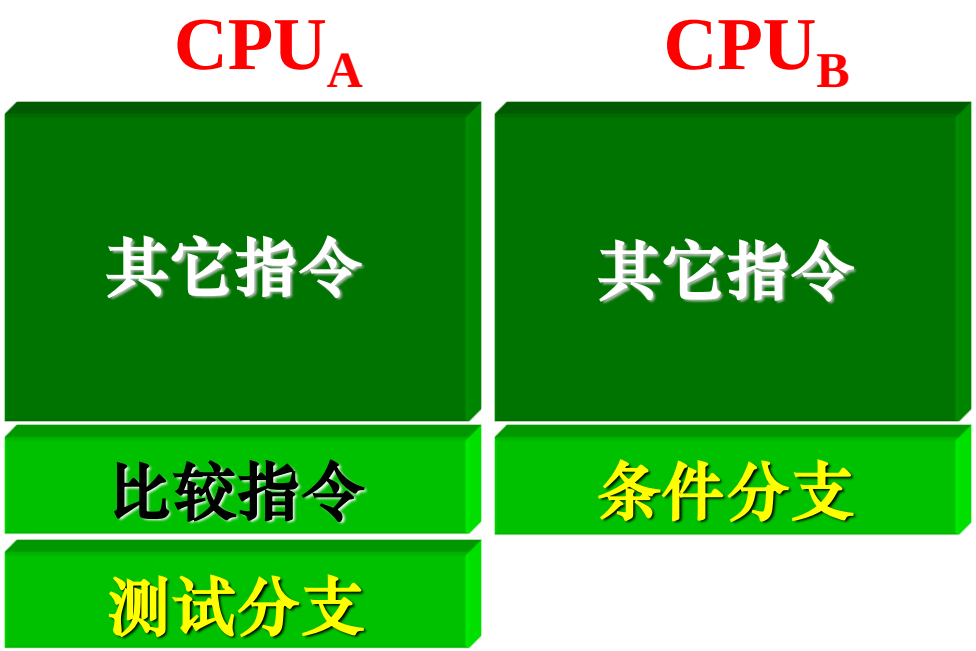
$$T_{CPU_A} = CPI_{CPU_A} \times IC_{CPU_A} \times T_{CLK_A}$$

$$T_{CPU_B} = CPI_{CPU_B} \times IC_{CPU_B} \times T_{CLK_B}$$

CPU_A	CPU_B
比较指令	条件分支
测试分支	

CPU性能公式

在两种CPU中，分支指令都占用2个时钟周期而所有其它指令占用1个时钟周期。

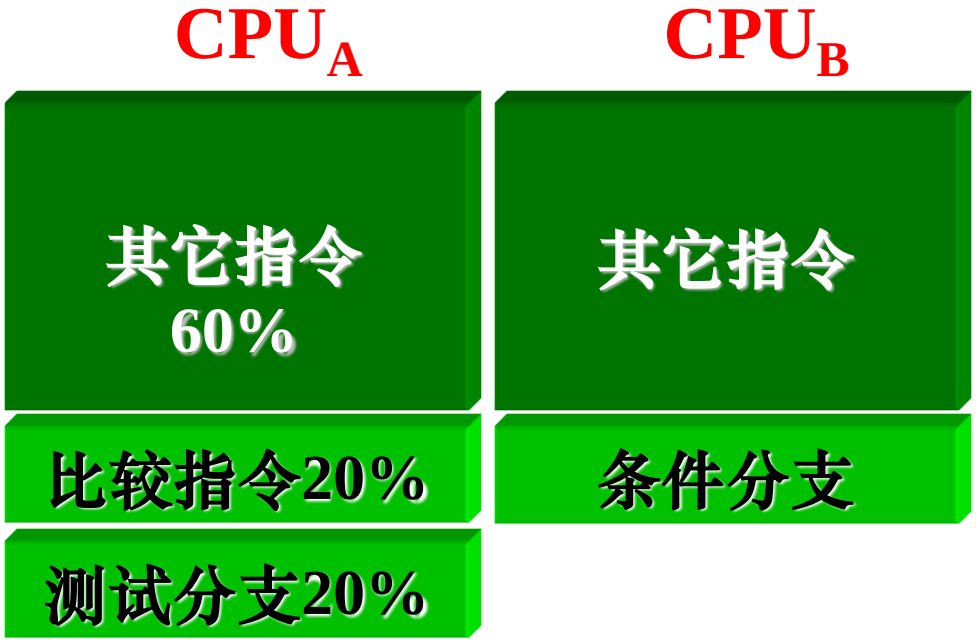


$$CPI_{分支} = 2$$

$$CPI_{其它} = 1$$

CPU性能公式

对于CPU_A，执行的指令中分支指令占20%；由于每个分支指令之前都需要有比较指令，因此比较指令也占20%。



$$CPI_{分支} = 2$$

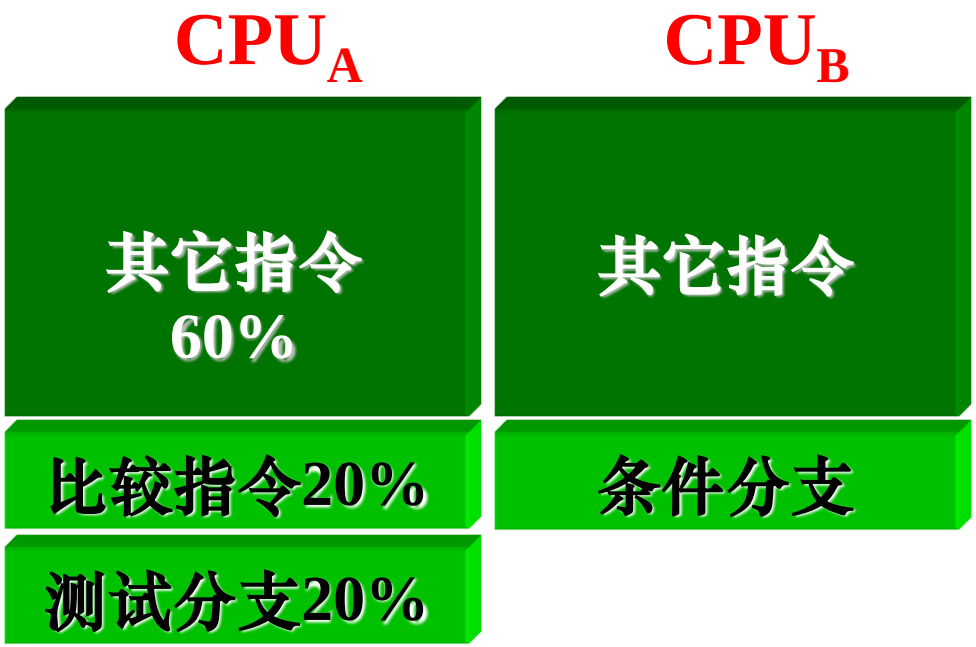
$$CPI_{其他} = 1$$

$$IC_B = （1 - 20\%）IC_A$$

$$IC_{B分支} = IC_{A分支}$$

CPU性能公式

由于CPU_A在分支时不需要比较，假设它的时钟频率比CPU_B快1.25倍。



$$CPI_{分支} = 2$$

$$CPI_{其他} = 1$$

$$IC_B = （1 - 20\%）IC_A$$

$$IC_{B分支} = IC_{A分支}$$

$$T_{clk_B} = 1.25\, T_{clk_A}$$

CPU性能公式

$$T_{CPU} = \sum (IC_i \times CPI_i) \times T_{CLK}$$

CPU_A	CPU_B
其它指令 60%	其它指令
比较指令20%	条件分支
测试分支20%	

$$CPI_{分支} = 2$$

$$CPI_{其他} = 1$$

$$IC_B = （1 - 20\%）IC_A$$

$$IC_{B分支} = IC_{A分支}$$

$$T_{clk_B} = 1.25\, T_{clk_A}$$

CPU性能公式

$$T_{CPU} = \sum (IC_i \times CPI_i) \times T_{CLK}$$

条件：

$\boldsymbol{CPI}_{分支} = 2$ $\boldsymbol{CPI}_{其他} = \mathbf{1}$

$\mathrm{IC}_{\boldsymbol{B}} =$ （$\mathbf{1-20\%}$）$\mathrm{IC}_{\boldsymbol{A}}$ $\mathrm{IC}_{\boldsymbol{B}}{}_{分支} = \mathrm{IC}_{\boldsymbol{A}}{}_{分支}$

$\boldsymbol{T}_{\mathrm{clk}_{\boldsymbol{B}}} = \mathbf{1.25}\boldsymbol{T}_{\mathrm{clk}_{\boldsymbol{A}}}$

求解$\mathrm{CPU_A}$的执行时间：

$$\boldsymbol{T}_{\boldsymbol{CPU_A}} = (\mathbf{80\%} \times \mathbf{1} + \mathbf{20\%} \times \mathbf{2}) \times \boldsymbol{IC_A} \times \boldsymbol{T}_{\boldsymbol{CLK_A}}$$

$$= \boldsymbol{IC_A} \times \mathbf{1.2} \times \boldsymbol{T}_{\boldsymbol{CLK_A}}$$

CPU性能公式

$$T_{CPU} = \sum (IC_i \times CPI_i) \times T_{CLK}$$

条件：

$\boldsymbol{CPI}_{分支} = 2$　　$\boldsymbol{CPI}_{其他} = \mathbf{1}$

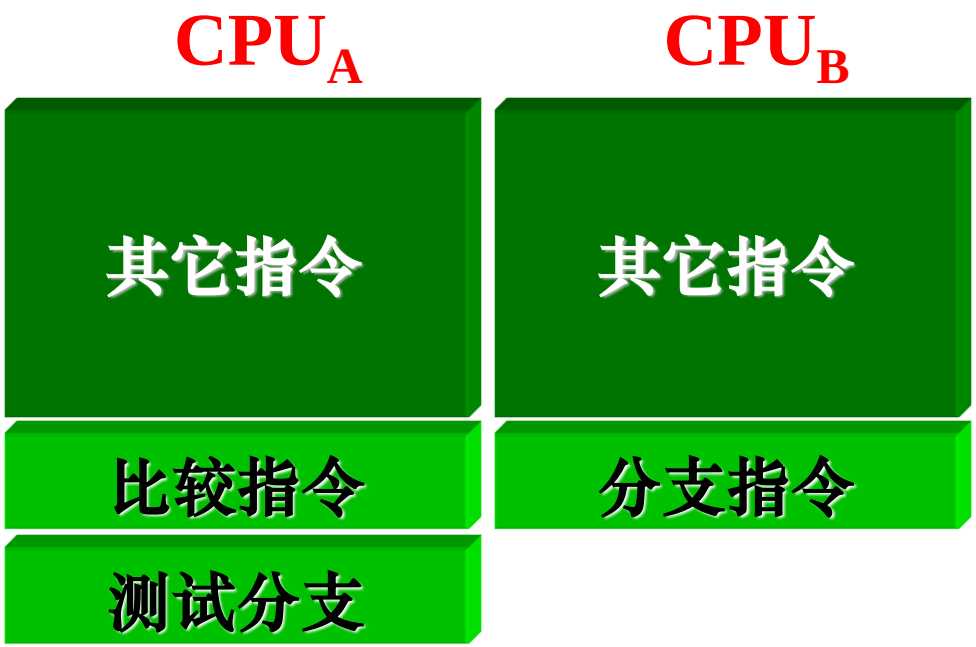
$\mathrm{IC}_{\boldsymbol{B}} = （\mathbf{1-20\%}）\mathrm{IC}_{\boldsymbol{A}}$　　$\mathrm{IC}_{\boldsymbol{B}}{}_{分支} = \mathrm{IC}_{\boldsymbol{A}}{}_{分支}$

$\boldsymbol{T}_{\mathrm{clk}_{\boldsymbol{B}}} = \mathbf{1.25}\boldsymbol{T}_{\mathrm{clk}_{\boldsymbol{A}}}$

求解$\mathrm{CPU_B}$的执行时间：

$$\begin{aligned} \boldsymbol{T}_{\boldsymbol{CPU_B}} &= (\mathbf{75\%} \times \mathbf{1} + \mathbf{25\%} \times \mathbf{2}) \times \boldsymbol{IC_B} \times \boldsymbol{T}_{\boldsymbol{CLK_B}} \\ &= \mathbf{1.25} \times \mathbf{0.8} \times \boldsymbol{IC_A} \times \mathbf{1.25} \times \boldsymbol{T}_{\boldsymbol{CLK_A}} \\ &= \boldsymbol{IC_A} \times \mathbf{1.25} \times \boldsymbol{T}_{\boldsymbol{CLK_A}} \end{aligned}$$

CPU性能公式



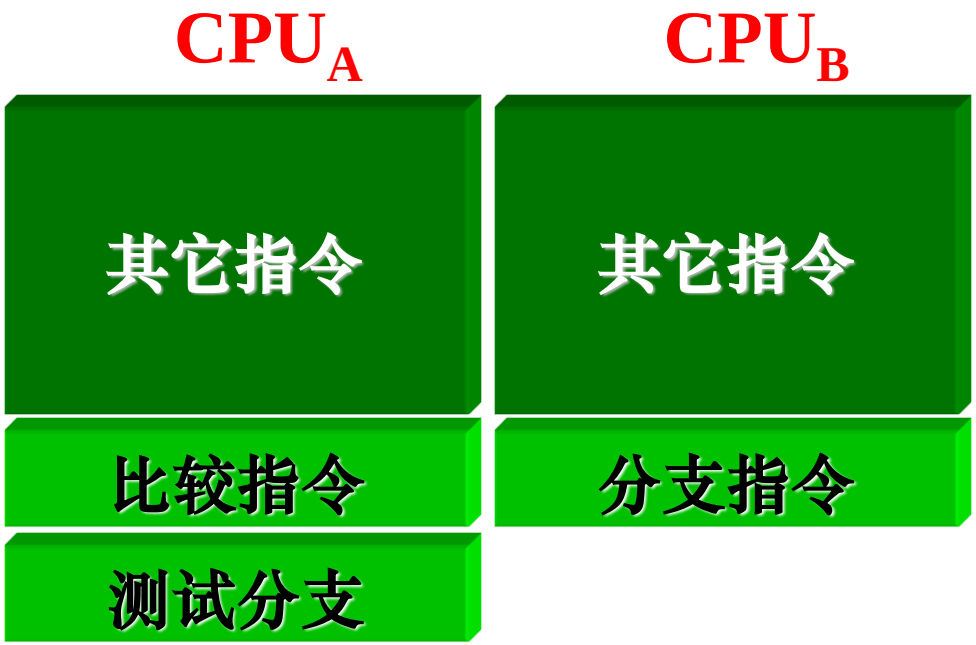
$$T_{CPU_A} = 1.2 \times IC_A \times T_{CLK_A}$$

$$T_{CPU_B} = 1.25 \times IC_A \times T_{CLK_A}$$

CPU$_A$快，即使其执行的指令数较CPU$_B$多。

CPU性能公式

- 如果CPUA的时钟周期时间仅仅比CPUB快1.1倍，哪一个CPU更快呢？



$$T_{CPU_A} = 1.2 \times IC_A \times T_{CLK_A}$$

$$T_{CPU_B} = 1.1 \times IC_A \times T_{CLK_A}$$

本章小结

- 讨论计算机体系结构的基本概念
  - 经典计算机体系结构概念
  - 计算机系统层次结构概念
  - 计算机组成和计算机实现技术
  - 现代计算机体系结构所研究的范围和内容
- 计算机体系结构的发展
  - 计算机的分代和分型
  - 计算机应用需求和实现技术等方面的发展对计算机体系结构发展的促进作用

本章小结

- 并行性技术提高计算机系统性能
  - 并行性技术的基本概念、分类、实例
  - 时间重叠、资源重复、资源共享
- 影响体系结构设计的成本和价格因素
  - 加深对计算机体系结构技术的理解
- 定量分析技术
  - 大概率事件优先原则
  - **Amdahl**定律
  - 程序的局部性原理
- 性能公式

第一章作业

- 王志英教材，P42，T5，T7

- 阅读：

[1] Y. Patt. Requirements Bottlenecks and Good Fortune Agents for Microprocessor Evolution, IEEE 2001.

[2] G. Amdahl. Architecture of the IBM System/360, IBM Journal, 1964

课程报告

以下报告内容均需要引用本课程的相关内容：

- 选择你认为计算机发展历史上最重要的十台机器，并说明理由。
- 由于AI的广泛应用，CPU+GPU的架构越来越普遍，比较各厂家架构设计上的特点。
- 近五年来，在CPU、存储、总线、IO 等方面有什么重要的新进展，分析它的影响和未来的发展。

第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

指令系统在计算机中的地位

2.1 指令系统概述

指令系统是计算机系统中软件与硬件交界的一个主要标志：

- 软件设计人员利用指令系统编制各种应用软件和系统软件；

- 硬件设计人员采用各种手段实现指令系统；

2.1 指令系统概述

- 指令集：
  - 一些指令的集合；
  - 每条指令都是直接由CPU硬件执行。
- 指令的表示方法：
  - 二进制格式；
  - 物理存储空间组织方式是位、字节、字和多字等；
  - 当前的指令字长有：16、32、64位；
  - 可变长格式和固定长度格式。

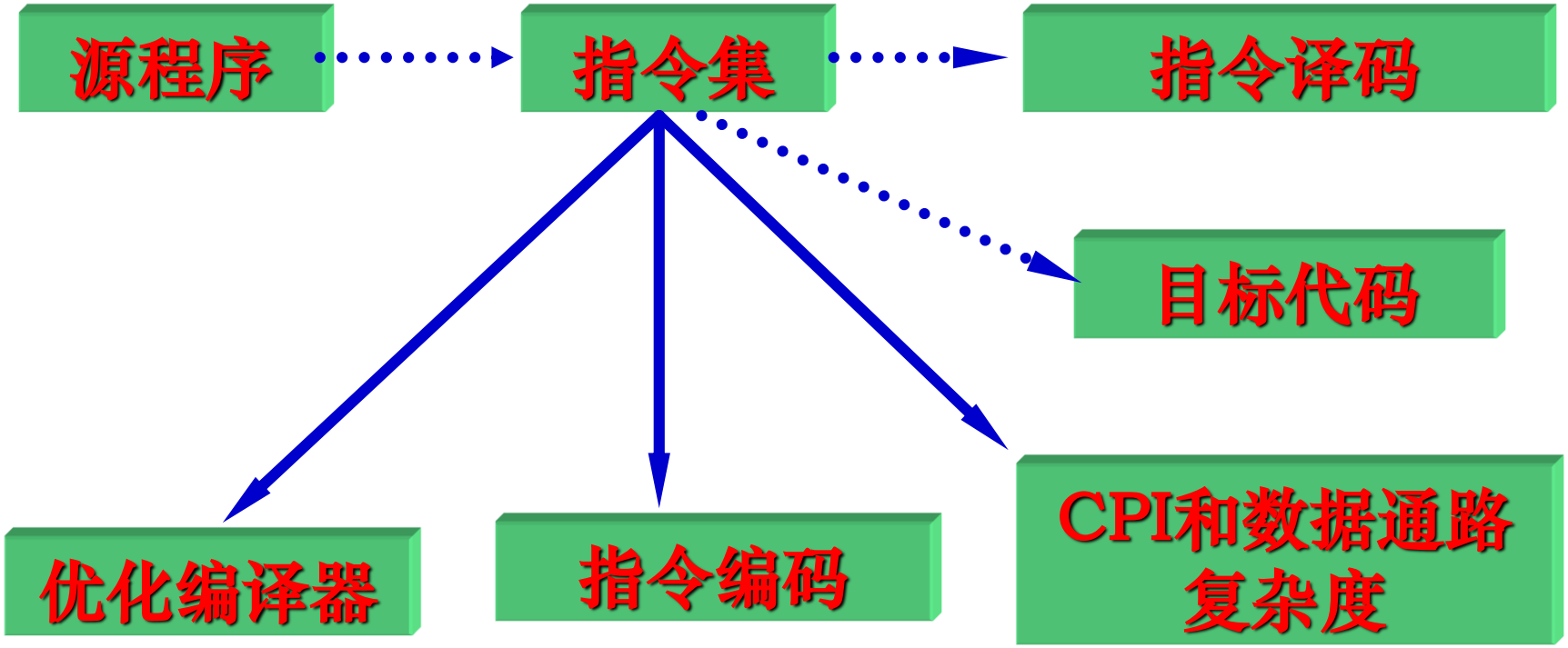
操作码	寻址方式	操作数	寻址方式	操作数

2.1 指令系统概述

- 指令的操作十分简单，其操作由操作码编码表示。
- 每个操作需要的操作数个数为0-3个不等。
  - 操作数是一些存储单元的地址；
  - 典型的存储单元通常有：主存、寄存器、堆栈和累加器。
- 操作数地址隐含表示或显式表示。

指令系统与计算机的性能

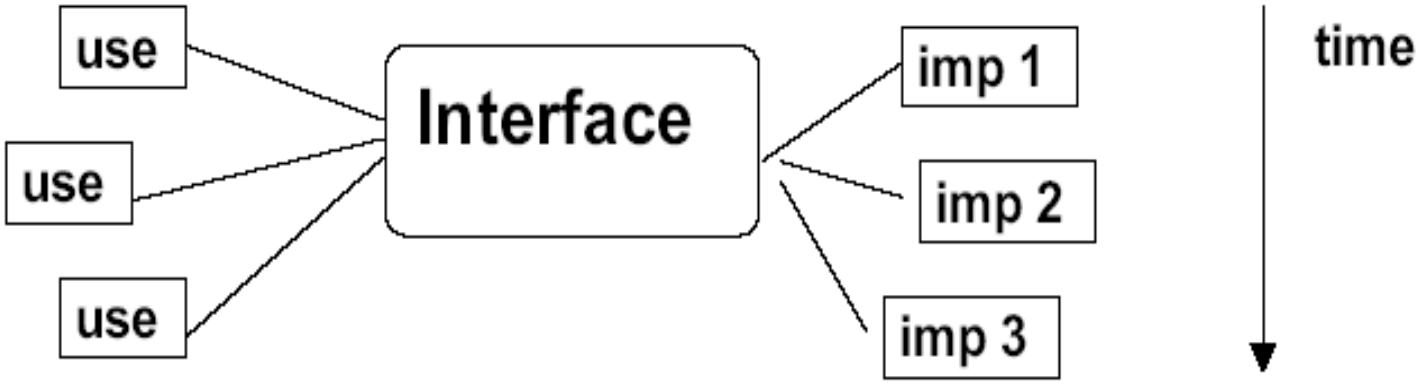
$$T_{CPU} = CPI \times IC \times T_{CLK}$$



2.1 指令系统概述

好的指令系统应该具备的条件

- 面向应用设计
- 方便高层软件的功能实现！
- 为底层硬件的高效率实现提供方便！



第2章 指令系统

2.1 指令系统概述

2.2 指令系统结构的分类

2.3 寻址方式

2.4 操作数类型和大小

2.5 指令系统的设计与优化

2.6 指令系统的发展和改进

2.7 指令格式举例

2.8 MIPS的一种简单实现

2.2 指令集结构的分类

- 一般来说，可以从如下五个因素考虑对计算机指令集结构进行分类，即：
  - 在CPU中操作数的存储方法；
  - 指令中显式表示的操作数个数；
  - 操作数的寻址方式；
  - 指令集所提供的操作类型；
  - 操作数的类型和大小。

2.2 指令集结构的分类

- CPU中用来存储操作数的存储单元主要有：
  - 堆栈；
  - 累加器；
  - 一组寄存器。
- 指令中的操作数可以被明确地显式给出，也可以按照某种约定隐式地给出。

2.2 指令集结构的分类

- Z=X+Y表达式在这三种类型指令集结构上的实现方法

堆栈	累加器	寄存器 (寄存器-存储器)	寄存器 (寄存器-寄存器)
PUSH X PUSH Y ADD POP Z	LOAD X ADD Y Store Z	LOAD R1,X ADD R1,Y Store R1,Z	LOAD R1,X LOAD R2,Y ADD R3,R1,R2 Store R3,Z

2.2 指令集结构的分类

- 早期的大多数机器都是采用堆栈型或累加器型指令集结构，但是自1980年以来的大多数机器均采用的是寄存器型指令集结构。主要有三个方面的原因:
  - 集成电路技术飞速发展
  - 寄存器和CPU内部其它存储单元一样，要比存储器快
  - 对编译器而言，可以更容易有效地分配和使用寄存器

通用寄存器型指令集结构

- 通用寄存器型指令集结构的主要优点:
  - 在表达式求值方面，比其它类型指令集结构都具有更大的灵活性；
  - 寄存器可以用来存放变量；
    - 减少存储器的通信量，加快程序的执行速度（因为寄存器比存储器快）
    - 可以用更少的地址位来寻址寄存器，从而可以有效改进程序的目标代码大小。

通用寄存器型指令集结构

- 两种主要的指令特性能够将通用寄存器型指令集结构（**GPR**）进一步细分。
  - **ALU**指令到底有两个或是三个操作数？
  - 在**ALU**指令中，有多少个操作数可以用存储器来寻址，也即有多少个存储器操作数？

通用寄存器型指令集结构

ALU指令中存储器操作数的个数	ALU指令中操作数的最多个数	结构类型	机器实例
0	3	RR	MIPS, SPARC, Alpha, PowerPC, ARM
1	2	RM	IBM 360/370, Intel 80x86, Motorola 6800
	3	RM	IBM 360/370
2	2	MM	VAX
3	3	MM	VAX

通用寄存器型指令集结构的分类

- 可以将当前大多数通用寄存器型指令集结构进一步细分为三种类型：
  - 寄存器－寄存器型
    (R－R：register-register)
  - 寄存器－存储器型
    (R－M：register-memory)
  - 存储器－存储器型
    (M－M：memory-memory)

(m，n)表示指令的n个操作数中有m个存储器操作数

三种通用寄存器型指令集结构的优缺点

- 寄存器－寄存器型（0,3）
  - 优点：
    - 指令字长固定，指令结构简洁，是一种简单的代码生成模型，各种指令的执行时钟周期数相近。
  - 缺点：
    - 与指令中含存储器操作数的指令系统结构相比，指令条数多，目标代码不够紧凑，因而程序占用的空间比较大。

三种通用寄存器型指令集结构的优缺点

- 寄存器－存储器型（1,2）
  - 优点：
    - 可以在ALU指令中直接对存储器操作数进行引用，而不必先用load指令进行加载，容易对指令进行编码，目标代码比较紧凑。
  - 缺点：
    - 由于有一个操作数的内容将被破坏，所以指令中的两个操作数不对称。在一条指令中同时对寄存器操作数和存储器操作数进行编码，有可能限制指令所能够表示的寄存器个数。指令的执行时钟周期因操作数的来源（寄存器或存储器）的不同而差别比较大。

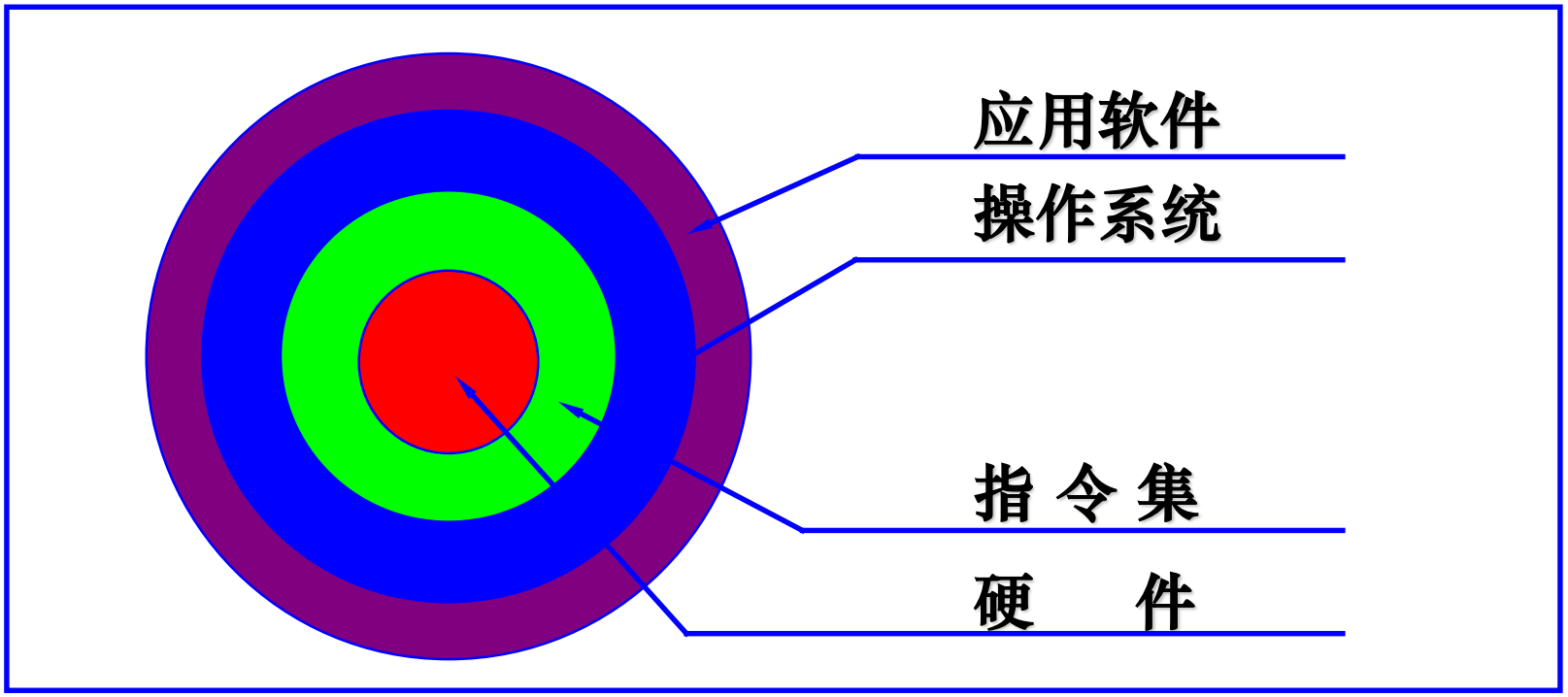
三种通用寄存器型指令集结构的优缺点

- 存储器－存储器型（（2，2）或（3，3））
  - 优点：
    - 目标代码最紧凑，不需要设置存储器来保存变量。
  - 缺点：
    - 指令字长变换很大，特别是3个操作数指令。而且每条指令完成的工作也差别很大。对存储器的频率访问会使存储器成为瓶颈。这种类型的指令系统现在已经不用了。

三种类型指令集结构的优缺点

指令集结构类型	优点	缺点
寄存器-寄存器（0，3）	指令字长固定，指令结构简洁，是一种简单的代码生成模型，各种指令的执行时钟周期数相近	与指令中含存储器操作数的指令系统结构相比，指令条数多，目标代码不够紧凑，因而程序占用的空间比较大
寄存器-存储器型（1，2）	可以在ALU指令中直接对存储器操作数进行引用，而不必先用load指令进行加载，容易对指令进行编码，目标代码比较紧凑	由于有一个操作数的内容将被破坏，所以指令中的两个操作数不对称。在一条指令中同时对寄存器操作数和存储器操作数进行编码，有可能限制指令所能够表示的寄存器个数。指令的执行时钟周期因操作数的来源（寄存器或存储器）的不同而差别比较大
存储器-存储器（2，2）或（3，3）	目标代码最紧凑，不需要设置存储器来保存变量	指令字长变换很大，特别是3个操作数指令。而且每条指令完成的工作也差别很大。对存储器的频率访问会使存储器成为瓶颈。这种类型的指令系统现在已经不用了

指令集结构设计概观



操作码	寻址方式	操作数	寻址方式	操作数